Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Mode Pengalamatan

Pertemuan 6

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 2.2

Menganalisis mode pengalamatan (*immediate*, *direct*, *register indirect*, *indexed*, *base-relative*, dan *based-indexed*) serta menentukan segmen *default* yang digunakan setiap mode.

CPMK yang didukung: CPMK02 ▪ **CPL yang didukung: CPL02**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan peran mode pengalamatan dalam menentukan letak operand;
2. mengenali mode pengalamatan dari penulisan operandnya;
3. menghitung alamat efektif dan alamat fisik operand;
4. menentukan segmen *default* dan *override prefix*;
5. memilih mode paling efisien untuk array dan *record*;
6. menuliskan satu operasi yang sama dalam beberapa mode.

Peta Materi

1. **Operand dan Tiga Mode Pengalamatan Dasar**
   opcode dan operand; mengapa mode pengalamatan diperlukan; mode *immediate*, *register*, dan *direct*

2. **Mode Memori Berpenunjuk dan Segmen Default**
   register indirect, *indexed*, *base-relative*, *based-indexed*; alamat efektif; kombinasi register legal; segmen `DS`/`SS` dan *override*

3. **Perbandingan Efisiensi dan Studi Kasus**
   jumlah byte dan siklus; satu operasi dalam tujuh mode; akses array dan medan *record*

Praktik pada pertemuan ini

Menelusuri `mode-array.asm` dan `param-bp.asm` dengan *debugger*, lalu memverifikasi alamat efektif pada jendela memori.

BAGIAN

Operand dan Tiga Mode Pengalamatan Dasar

Operand, Opcode, dan Mengapa Mode Diperlukan

- Setiap instruksi mesin tersusun atas **opcode** dan **operand**. Pada `MOV AX, BX`, `AX` adalah operand tujuan dan `BX` operand sumber.
- Pertanyaan pokok prosesor: *di manakah operand itu berada?* Data dapat berada di dalam instruksi, di dalam register, atau di dalam memori pada alamat tertentu.
- Cara prosesor menyatakan letak operand disebut **mode pengalamatan** (*addressing mode*).
- Mode pengalamatan menjadi jembatan antara penulisan simbolik programmer dan byte *ModR/M* hasil assembler. Karena byte itu hanya delapan bit, jumlah cara yang dapat dikodekan memang terbatas.

Batasan penting

Operand tujuan tidak boleh berupa nilai segera: `MOV 5, AX` tidak mungkin, karena angka 5 bukan tempat yang dapat menampung hasil.

Sumber: Bab VIII, Mengapa Mode Pengalamatan Diperlukan

Mode Immediate

Operand sumber adalah nilai konstan yang sudah tertulis di dalam instruksi, sehingga prosesor tidak perlu mengakses memori.

```
MOV AX, 1234h        ; AX <- 1234h   (immediate 16 bit)
MOV BL, 0Ah          ; BL <- 0Ah     (immediate 8 bit)
ADD CX, 100          ; CX <- CX + 100
MOV AL, 'A'          ; AL <- 41h, kode ASCII huruf 'A'
```

Ciri dan kegunaan

Ukuran nilai segera mengikuti ukuran register tujuan: 8 bit untuk `AL` dan 16 bit untuk `AX`. Nilai segera tidak mengenal segmen karena tidak berada di memori, dan dipakai untuk mengisi nilai awal serta menaikkan nilai dengan langkah tetap.

Sumber: Bab VIII, Mode Immediate

Mode Register

Operand berada di dalam register prosesor; baik sumber maupun tujuan berupa register, tanpa pengaksesan memori.

```
MOV AX, BX      ; AX <- BX
ADD DX, CX      ; DX <- DX + CX
MOV DL, AL      ; DL <- AL (ukuran operand harus sama)
```

Tercepat dan paling ringkas

`MOV AX, BX` hanya menempati 2 byte dan selesai dalam dua siklus. Keterbatasannya adalah jumlah register 8086 yang sangat sedikit.

Sumber: Bab VIII, Mode Register

Mode Direct (Memori Langsung)

Operand berada di memori dan alamatnya (offset) ditulis harfiah di dalam instruksi.

```
.DATA
    nilai   DW  1234h
    total   DW  ?
.CODE
    MOV AX, nilai       ; AX <- isi memori DS:nilai
    MOV AX, [1234h]     ; AX <- isi memori pada offset 1234h
    MOV total, AX       ; simpan AX ke variabel total
```

Alamat pada mode ini tetap selamanya karena tertulis di dalam instruksi, sehingga cocok untuk variabel tunggal. Kerugiannya, setiap variabel memerlukan byte kode sendiri dan datanya tidak dapat dipindahkan tanpa dirakit ulang.

Tiga Mode Pengalamatan Dasar

Mode	Bentuk penulisan	Contoh	Akses memori	Kegunaan utama
Immediate	nilai konstan	`MOV AX, 1234h`	tidak	mengisi nilai awal dan tetapan
Register	register	`MOV AX, BX`	tidak	memindahkan data antarvariabel kerja
Direct	`[disp]` atau nama variabel	`MOV AX, [1234h]`	ya, sekali	variabel tunggal beralamat tetap

Sumber: Bab VIII, Mode Direct

Tampak Mirip, Artinya Berbeda

- `MOV AX, nilai` menyalin *isi* variabel `nilai` — mode *direct*.
- `MOV AX, OFFSET nilai` menyalin *offset*-nya sebagai nilai segera — mode *immediate*.
- `MOV AX, 1234h` memuat nilai `1234h`, sedangkan `MOV AX, [1234h]` memuat isi memori pada offset itu.

Segmen *default* mode direct

Offset pada mode *direct* dianggap berada pada segmen `DS` kecuali dinyatakan lain.

Ketiga mode dasar belum memanfaatkan register penunjuk, sehingga alamat memori harus tetap dan tidak dapat dihitung saat program berjalan.

Sumber: Bab VIII, Mode Direct

Rangkuman Bagian 1

- Instruksi mesin tersusun atas opcode dan operand; prosesor harus lebih dahulu tahu *di mana* operand berada.
- Mode pengalamatan adalah jembatan antara penulisan simbolik programmer dan byte *ModR/M*. Karena byte itu 8 bit, jumlah cara yang dapat dikodekan terbatas.
- Mode *immediate* menyimpan konstanta di dalam instruksi: tanpa akses memori dan tanpa segmen. Operand tujuan tidak boleh berupa nilai segera.
- Mode *register* paling cepat dan ringkas (2 byte, 2 siklus); mode *direct* memakai offset tetap pada segmen `DS` dan cocok untuk variabel tunggal.

BAGIAN

Mode Memori Berpenunjuk dan Segmen Default

Mode Register Indirect

Operand ditulis di antara tanda kurung siku berisi sebuah register; register itu menyimpan *offset* tempat data berada.

```
MOV BX, OFFSET data1    ; BX <- offset awal array data1
MOV AX, [BX]            ; AX <- isi memori DS:BX
ADD BX, 2               ; maju dua byte (satu elemen word)
MOV AX, [BX]            ; AX <- elemen kedua array
```

- Hanya `BX`, `BP`, `SI`, dan `DI` yang boleh menjadi penunjuk alamat.
- `BX` dan `BP` adalah **register basis**; `SI` dan `DI` adalah **register indeks**.

Segmen [BX] dan [BP] berbeda

`[BX]` memakai `DS`, sedangkan `[BP]` memakai `SS`; `MOV AX, [BP]` membaca *stack*, bukan segmen data.

Mode Indexed dan Base-Relative

Indexed: `[SI+disp]`, `[DI+disp]`

register indeks + perpindahan

```
MOV SI, 4
MOV AX, data1[SI]
MOV AX, [SI+8]
```

Base-relative: `[BX+disp]`, `[BP+disp]`

register basis + perpindahan

```
MOV AX, [BX+4]
MOV BP, SP
MOV AX, [BP+2]
```

`data1[SI]` diterjemahkan assembler menjadi `[data1 + SI]`, menyerupai penulisan array pada bahasa tingkat tinggi. Perpindahan boleh berukuran 8 bit, 16 bit, atau nol.

Sumber: Bab VIII, Mode Indexed dan Base-Relative

Mode Based-Indexed

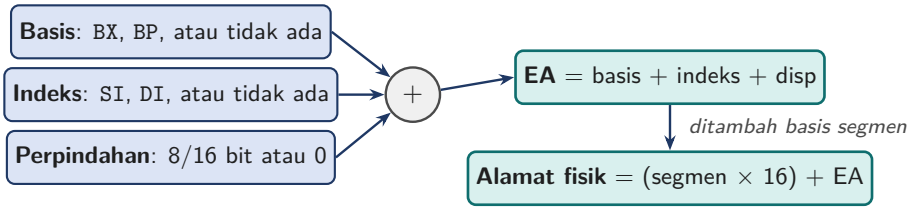
Menggabungkan satu basis, satu indeks, dan satu perpindahan sekaligus. Hanya empat kombinasi yang legal: `[BX+SI+disp]`, `[BX+DI+disp]`, `[BP+SI+disp]`, dan `[BP+DI+disp]`.

```
MOV BX, OFFSET tabel    ; basis  : awal tabel record
MOV SI, 6               ; indeks : record ke-3 (3 x 2 byte)
MOV AX, [BX+SI+2]       ; AX <- medan ke-2 dari record ke-3
```

Kapan dipakai

Bila alamat akhir tersusun dari dua komponen yang bergerak sendiri-sendiri, misalnya baris dan kolom matriks, atau nomor *record* dan nomor medannya.

Menghitung Alamat Efektif



Alamat efektif (*effective address*) adalah offset 16 bit yang kembali ke nol setelah melewati batas segmen; alamat fisik dibentuk dengan menambahkan basis segmen.

Sumber: Bab VIII, Menghitung Alamat Efektif

Contoh Perhitungan Alamat

Diketahui `DS` $=$ `1000h`, `BX` $=$ `0200h`, `SI` $=$ `0006h`, dan instruksi `MOV AX, [BX+SI+2]`.

Alamat efektif

`0200h` $+$ `0006h` $+$ `0002h` $=$ `0208h`

Alamat fisik

`1000h` $\times$ `10h` $+$ `0208h` $=$ `10208h`

Verifikasi

Alamat fisik inilah yang muncul pada jendela memori *debugger* saat program ditelusuri satu instruksi demi satu.

Sumber: Bab VIII, Menghitung Alamat Efektif

Kombinasi Register yang Legal dan Ilegal

Legal

- `[BX]`, `[SI]`, `[DI]` — DS
- `[BP]` — SS
- `[BX+SI]`, `[BX+DI]` — DS
- `[BP+SI]`, `[BP+DI]` — SS

Ditolak assembler

- `[AX]`, `[CX]`, `[DX]` — bukan penunjuk
- `[SP]` — SP bukan register basis
- `[BX+BP]` — dua register basis
- `[SI+DI]`, `[BX+SI+DI]` — dua indeks

Pesan `Invalid use of register` bukan kesalahan penulisan semata: 8086 memang tidak memiliki *encoding*-nya, sesuai struktur byte *ModR/M*.

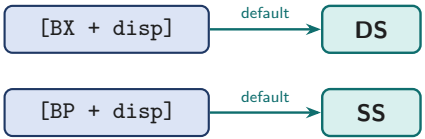
Sumber: Bab VIII, Kombinasi Register yang Legal dan Ilegal

Segmen *Default* Setiap Operand Memori

Bentuk operand	**Segmen**	**Keterangan dan override**
`[disp]`, nama variabel	`DS`	Override: `CS:`, `SS:`, atau `ES:`
`[BX]`, `[SI]`, `[DI]`, `[BX+disp]`	`DS`	Penunjuk data; override `SS:`, `ES:`, `CS:`
`[BP]`, `[BP+disp]`, `[BP+SI]`	`SS`	Penunjuk stack; override `DS:`, `ES:`, `CS:`
Operand string `DI` sebagai tujuan	`ES`	Tidak dapat di-override
Pengambilan instruksi (`IP`)	`CS`	Tidak dapat di-override
`PUSH` dan `POP`	`SS`	Selalu memakai stack; tidak dapat di-override

Sumber: Bab VIII, Segmen Default dan Segment Override

Segment Override Prefix



Bila tidak disebutkan, prosesor memakai segmen *default* yang bergantung pada register penunjuknya.

Override ditulis dengan menempatkan nama register segmen di depan tanda kurung:

```
MOV AX, ES:[BX]
```

Yang berubah dan yang tidak

Override hanya mengganti register segmen, bukan offset, dan menambah satu byte awalan (*prefix*).

Rangkuman Bagian 2

- Empat mode berpenunjuk menghitung alamat dari isi register, sehingga alamatnya dapat berubah saat program berjalan — dasar akses array dan *record*.
- Hanya `BX`, `BP`, `SI`, dan `DI` yang boleh menjadi penunjuk; `BX`/`BP` adalah basis, `SI`/`DI` adalah indeks.
- `EA` = `base` + `index` + `disp`, lalu alamat fisik = (segmen $\times$ 16) + EA.
- `[BX]` dan `[SI]` memakai `DS`, sedangkan `[BP]` memakai `SS` — itulah sebabnya `BP` dipakai untuk membaca parameter pada *stack*.
- *Segment override prefix* seperti `ES:` mengganti register segmen, bukan offset, dan menambah satu byte awalan.

BAGIAN

Perbandingan Efisiensi dan Studi Kasus

Ringkasan Tujuh Mode Pengalamatan

Mode	Bentuk penulisan	Contoh	Segmen default	Byte
Immediate	nilai konstan	`MOV AX, 1234h`	tidak ada	3
Register	register	`MOV AX, BX`	tidak ada	2
Direct	`[disp16]`	`MOV AX, [1234h]`	`DS`	3
Register indirect	`[BX]`, `[SI]`, `[DI]`	`MOV AX, [BX]`	`DS` (`BP`: `SS`)	2
Indexed	`[SI+disp]`, `[DI+disp]`	`MOV AX, [SI+8]`	`DS`	3
Base-relative	`[BX+disp]`, `[BP+disp]`	`MOV AX, [BX+8]`	`DS` (`BP`: `SS`)	3
Based-indexed	`[BX+SI+disp]` dan tiga kombinasi lain	`MOV AX, [BX+SI+8]`	`DS` (`BP`: `SS`)	3

Sumber: Bab VIII, Ringkasan Tujuh Mode Pengalamatan

Jumlah Byte dan Siklus Relatif

Mode	Contoh	Byte	Siklus	Paling cocok untuk
Register	`MOV AX, BX`	2	2	data kerja yang paling sering
Immediate	`MOV AX, 1234h`	3	4	nilai awal dan tetapan
Direct	`MOV AX, [1234h]`	3	10	variabel tunggal beralamat tetap
Register indirect	`MOV AX, [BX]`	2	13	penelusuran *pointer*
Indexed	`MOV AX, [SI+8]`	3	17	elemen array berindeks
Base-relative	`MOV AX, [BX+8]`	3	17	medan *record* dan `[BP+disp]`
Based-indexed	`MOV AX, [BX+SI+8]`	3	19	matriks dan array di dalam *record*

Sumber: Bab VIII, Jumlah Byte dan Siklus Relatif

Satu Operasi dalam Tujuh Mode

Menyalin elemen ke-2 array `data1` (nilai `1Eh`) ke register `AX`:

```
MOV AX, 30              ; 1. immediate
MOV BX, 30
MOV AX, BX              ; 2. register
MOV AX, data1+4         ; 3. direct
MOV BX, OFFSET data1
ADD BX, 4
MOV AX, [BX]            ; 4. register indirect
MOV SI, 4
MOV AX, data1[SI]       ; 5. indexed
MOV BX, OFFSET data1
MOV AX, [BX+4]          ; 6. base-relative
MOV BX, OFFSET data1
MOV SI, 4
MOV AX, [BX+SI]         ; 7. based-indexed
```

Ketujuhnya menghasilkan nilai yang sama, tetapi berbeda panjang kode dan jumlah siklusnya.

Studi Kasus: Elemen Array Satu Dimensi

Array *word* disimpan berurutan; elemen ke-i berada pada offset awal array ditambah $2i$.

```
MOV BX, OFFSET data1      ; BX = basis, awal array
XOR SI, SI                ; SI = 0, indeks elemen pertama
MOV CX, 5                 ; banyaknya elemen
ULANG:
MOV AX, [BX+SI]           ; AX <- data1[i]
; ... olah nilai AX di sini ...
ADD SI, 2                 ; maju ke elemen berikutnya
LOOP ULANG                ; ulangi sampai CX = 0
```

Bila elemen berukuran satu byte, langkahnya cukup `INC SI`; bila empat byte, `ADD SI, 4`. Yang berubah hanyalah besarnya langkah, bukan bentuk pengalamatannya.

Studi Kasus: Medan *Record* dengan Base-Relative

Record menyatukan beberapa medan bertipe berbeda dalam satu blok memori. Karena panjang medannya tetap, offsetnya pun tetap.

```
.DATA
    mhs1    DB  '2210'        ; medan NIM   : offset 0 sampai 3
    nilai1  DW  85            ; medan nilai : offset 4 sampai 5
.CODE
    MOV BX, OFFSET mhs1       ; basis = awal record
    MOV AX, [BX+4]            ; AX <- medan nilai (85)
```

Bentuk yang sama dengan BP

`[BP+disp]` dipakai untuk membaca parameter prosedur karena bawaannya menunjuk segmen SS.

Rangkuman Bagian 3

- Efisiensi menurun seiring bertambahnya komponen alamat: mode *register* 2 siklus, sedangkan *based-indexed* 19 siklus.
- *Register indirect* paling ringkas di antara mode memori (2 byte), tetapi tetap 13 siklus; *immediate* 3 byte dan 4 siklus.
- Satu operasi dapat dituliskan dalam tujuh mode; hasilnya sama, panjang kode dan jumlah siklusnya berbeda. Cara 4–7 lebih menguntungkan bila indeks berubah saat program berjalan.
- Array satu dimensi ditelusuri dengan satu instruksi tetap `[BX+SI]` yang indeksnya dinaikkan 2 tiap putaran.
- Pada *record*, basis menunjuk awal blok dan perpindahan memilih medannya: `[BX+4]` mengambil medan nilai.

Contoh 1: Tiga Mode pada Satu Array

Menjumlahkan lima elemen `data1` dengan tiga mode pengalamatan berbeda; ketiganya harus memberi jumlah yang identik.

Cara 1: direct

```
MOV AX, data1+0
ADD AX, data1+2
ADD AX, data1+4
ADD AX, data1+6
ADD AX, data1+8
MOV jumlah1, AX
```

Cara 2: indexed

```
XOR SI, SI
MOV CX, n
C2: ADD AX, data1[SI]
    ADD SI, 2
    LOOP C2
MOV jumlah2, AX
```

Cara 3: base-relative

```
LEA BX, data1
XOR SI, SI
MOV CX, n
C3: ADD AX, [BX+SI]
    ADD SI, 2
    LOOP C3
MOV jumlah3, AX
```

Keluaran

```
Perbandingan tiga mode
Ketiga hasil identik.
```

`CMP` dan `JNE` membandingkan `jumlah1` dengan `jumlah2`, lalu `jumlah2` dengan `jumlah3`. Ketiganya bernilai $10 + 20 + 30 + 40 + 50 = 150$ (96h). Sumber: `mode-array.asm`.

Contoh 2: Membaca Parameter dengan [BP+disp]

```
MOV AX, 0002h     ; parameter kedua
PUSH AX
MOV AX, 0001h     ; parameter pertama
PUSH AX
MOV BP, SP        ; basis bingkai stack
MOV AX, [BP+0]    ; param. 1 -> SS:BP + 0
MOV BX, [BP+2]    ; param. 2 -> SS:BP + 2
ADD AX, BX        ; AX = 1 + 2 = 3
MOV DL, AL        ; hasil -> DL
ADD DL, '0'       ; angka -> karakter
MOV AH, 02h       ; cetak satu karakter
INT 21h
POP AX            ; bersihkan stack
POP AX
```

Keluaran

```
1 + 2 = 3
```

8086 tidak memiliki `PUSH 2` untuk nilai segera, sehingga nilainya dimuat ke `AX` lebih dahulu.

`[BP+0]` dan `[BP+2]` memakai segmen `SS` secara bawaan; tanpa `BP`, `[BX+2]` akan membaca lokasi lain pada `DS`.

`param-bp.asm`

Praktikum: Menelusuri Mode Pengalamatan

Judul praktikum

Penelusuran mode pengalamatan dan verifikasi alamat efektif dengan *debugger* ▪ **Sub-CPMK:** 2.2 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu

Alat dan bahan

- Komputer bersistem operasi Windows
- Emu8086 (tampilan register dan memori)
- DOSBox beserta `DEBUG`, TASM, dan TLINK
- Berkas `mode-array.asm` dan `param-bp.asm`
- Lembar kerja pengamatan alamat efektif

Langkah kerja

1. Rakit kedua berkas contoh dengan TASM dan TLINK.
2. Aktifkan `Single Step`, catat `DS`, `BX`, `SI`, `BP`, lalu hitung EA-nya.
3. Jalankan `debug mode-array.exe` dan pakai perintah `u`, `t`, `d`.
4. Ubah `[BP+0]` menjadi `[BX+0]`, jelaskan perubahan keluarannya.

Dikumpulkan: lembar pengamatan alamat efektif, tangkapan layar Emu8086 dan sesi `DEBUG`, serta lembar jawaban Kuis 2.

Latihan

1. Jelaskan mengapa mode pengalamatan diperlukan, padahal satu instruksi `MOV` sudah cukup untuk memindahkan data. Kaitkan dengan keterbatasan byte *ModR/M*.

2. Tentukan mode pengalamatan operand sumber dan segmen *default*-nya untuk: `MOV AX, 1234h`; `MOV AX, CX`; `MOV AX, nilai`; `MOV AX, [BX]`; `MOV AX, [BP+2]`; `MOV AX, data1[SI]`; `MOV AX, [BX+DI+6]`; `MOV AX, ES:[BX]`.

3. Diketahui `DS` = `1234h`, `SS` = `2000h`, `BX` = `0050h`, `BP` = `0010h`, `SI` = `0008h`, `DI` = `0002h`. Hitung alamat efektif dan alamat fisik `MOV AX, [SI+4]`, `MOV AX, [BP+DI]`, dan `MOV AX, [BX+SI+2]`.

4. Manakah penulisan berikut yang ditolak assembler? Jelaskan alasan setiap penolakan: `[AX]`, `[BX+BP]`, `[SI+DI]`, `[BX+SI+DI]`, `[BP+DI]`, `[SP+2]`.

5. Urutkan `MOV AX, BX`, `MOV AX, [BX]`, `MOV AX, [BX+SI+2]`, dan `MOV AX, [1234h]` dari yang paling ringkas ke yang paling panjang. Sertakan perkiraan byte dan siklusnya, lalu jelaskan mengapa urutannya demikian.

Rangkuman

- **Mode pengalamatan** adalah cara prosesor menyatakan letak operand — jembatan antara penulisan simbolik programmer dan byte *ModR/M* hasil assembler.
- Tujuh mode 8086: *immediate*, *register*, *direct*, *register indirect*, *indexed*, *base-relative*, dan *based-indexed*.
- *Immediate* dan *register* tidak mengakses memori sehingga paling cepat; *direct* memakai offset tetap; empat mode sisanya membentuk alamat dari register penunjuk.
- `EA` = `base` + `index` + `disp`; alamat fisik = (segmen $\times$ 16) + EA.
- Penunjuk alamat hanya `BX`, `BP`, `SI`, dan `DI`; `[AX]`, `[BX+BP]`, `[SI+DI]`, dan `[BX+SI+DI]` ditolak karena tidak ada *encoding*-nya.
- Segmen *default*: `DS` untuk `BX`/`SI`/ `DI` dan `SS` untuk `BP`. *Override prefix* mengubah segmen tanpa mengubah offset.

Cek Pemahaman

1. Mode pengalamatan yang dipakai oleh operand `MOV AX, [BX+SI+4]` adalah ...
   1.1 *indexed*
   1.2 *base-relative*
   1.3 *based-indexed*
   1.4 *register indirect*

2. Segmen *default* untuk operand `MOV AX, [BP+2]` adalah ...
   2.1 `CS`
   2.2 `DS`
   2.3 `ES`
   2.4 `SS`

3. Penulisan operand berikut yang ditolak assembler karena tidak memiliki *encoding* 8086 adalah ...
   3.1 `MOV AX, [SI+DI]`
   3.2 `MOV AX, [BX+DI]`
   3.3 `MOV AX, [BP+DI]`
   3.4 `MOV AX, [BX+SI]`

Penutup

Sebelum pertemuan berikutnya

- Selesaikan praktikum: lembar pengamatan alamat efektif, tangkapan layar Emu8086 dan `DEBUG`, serta lembar jawaban Kuis 2.
- Ulangi bagian yang belum dikuasai dengan menelusuri `mode-array.asm` satu instruksi demi satu.
- Baca **Bab IX** — Direktif Assembler dan Alur Translasi Program.

Bacaan lanjutan: Bab VIII bagian Segmen Default dan Segment Override; daftar *encoding* dan waktu siklus Intel 8086; pembahasan segmen secara mendalam pada Bab VI.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil